CHRF++-words helping character n-grams带读

给字符 n-gram 打补丁，让自动评估更贴近人类打分

0. TL;DR（≤120字）

- 贡献一句话：在 chrF（character n-gram F-score）上加入 word unigram/bigram 得到 chrF++，与 DA（direct assessment）相关性更稳。
- 适用一句话：低资源/形态变化多/拼写多样的 MT（machine translation）评估与调参，尤其适合做验证集主指标和解码超参搜索。
- 风险一句话：偏 recall 的指标（β>1）可能诱导输出变长；同时“word”粒度依赖空白/分词边界，可能带来跨数据源抖动。

1. 背景与动机：他们到底在“卡什么”

任务背景：<用 5~10 句讲清语境>

机器翻译里，人评（human evaluation）贵且慢，工程上常用自动指标（automatic metric）做模型选择与调参。
BLEU 这类词级匹配（word-level matching）对同义改写、形态变化（morphology）、轻微拼写差异很苛刻；chrF 则用字符 n-gram（character n-gram）做匹配，语言无关、对形态变化更鲁棒。
WMT 的评测机制也在演进：早期偏向 RR（relative rankings，相对排序），后来引入并逐步转向 DA（direct assessment，绝对打分），后者更直接反映译文的“绝对质量”。
作者的现实问题是：chrF 在 RR 环境下表现很好，但在 DA 环境下是否仍然最优？如果不够，该怎么改，才能更贴近 DA 的统计关系并更好地服务系统调参？

痛点拆解：<为什么已有方法不够>

作者用散点图直观展示了一个工程痛点：

- WORDF（word n-gram F-score）在“人类觉得很好”的译文区域经常过度悲观（overly pessimistic）：DA 很高但 WORDF 分偏低。
- chrF 更集中、更贴 DA，但可能存在“略乐观（too optimistic）”的情况：chrF 高但 DA 低（例如字符相似但词选择/短语搭配不自然）。
  换句话说：词级太严格、字符级太宽松，二者各有盲区。

作者的关键洞见：<一句话点题>

把“词级的短依赖”（word unigram/bigram）加回 chrF，让字符级匹配不至于“自嗨”，从而在 DA 上更稳、更像人。

2. 问题定义与符号（足够让人复现）

输入/输出：<清晰列出>

- 输入（hypothesis / system output）：系统生成的译文字符串（raw text）
- 参考（reference / gold translation）：一条或多条参考译文（multiple references）
- 输出：
  - 句级（segment-level）分数：chrF / chrF+ / chrF++
  - 文档/语料级（document-level）分数：支持 micro / macro 汇总（具体定义以脚本为准）

> 重要工程口径：作者强调脚本面向 raw text、语言无关，不强制依赖 tokenisation（分词）或额外工具链。

> 但工程上仍建议统一 Unicode 与空白规则，避免评测抖动（见第 5 节）。

目标函数/评价指标

作者用统一的 n-gram Fβ 框架（ngrFβ）描述 chrF 与 WORDF，并用 Pearson correlation（Pearson's r）在句级对齐 DA 来评估"指标好坏"。

Fβ 公式：

$$\mathrm{F}_\beta = \frac{(1+\beta^2)\cdot P\cdot R}{\beta^2\cdot P + R}$$

其中：

- P（precision）：hypothesis 中 n-gram 有多少能在 reference 中匹配到（通常按 clipped count 计数）
- R（recall）：reference 中 n-gram 有多少在 hypothesis 中出现
- β：控制 recall 相对 precision 的权重（β>1 更偏召回）

对 chrF / WORDF 来说，P, R 不是单一 n，而是对 $n = 1..N$ 的算术平均（arithmetically averaged）后的整体 P, R（实现细节见第 4 节伪代码）。

关键假设：<作者默认了什么>

1. DA（direct assessment）比 RR 更能代表"绝对质量"，因此用 DA 做相关性验证是合理的工程目标。
2. chrF 的优势来自字符级鲁棒性；WORDF 的劣势主要来自较长的 word n-gram（3/4-gram）对改写太敏感。
3. 经验上 WORDF 取 N=4、chrF 取 N=6 足够；继续增大 N 收益有限。

3. 方法总览：一张"脑内结构图"

用文字 + 列表给出整体流水线（可以 ASCII 流程图）

```
(hypothesis, reference[s])
        |
        v
[raw text] --(optional normalize)──> [normalized text]
        |
        +──> Character n-grams (n=1..6) ──> P_char, R_char (avg over n)
        |
        +──> Word n-grams (n=1..2)      ──> P_word, R_word (avg over n)
        |
        v
Combine char + word stats (average together)
        |
        v
F_beta (beta=2 default)
        |
        +──> segment-level score
        +──> document-level score (micro / macro)
```

标出每一步的"作用"和"替代方案"

- 字符 n-gram（character n-grams）
  - 作用：对形态变化、拼写差异更鲁棒；降低"同义改写被错杀"
  - 替代：BLEU / 纯 word n-gram / 学习型指标（如 COMET 类，但会更重、更复杂）
- 短词 n-gram（word unigram/bigram）

    - 作用：补“词选择”和“局部搭配/短依赖”，抑制 chrF 的潜在乐观偏差
    - 替代：只加 unigram（chrF+）；或回到 METEOR 等规则/词形工具更重的指标
- β 的选择（beta=2）
    - 作用：在保持相关性提升的同时，降低“偏 recall 导致输出变长”的副作用风险
    - 替代：β=1（更偏 precision）/ β=3（更偏 recall，但更可能诱导过长）

4. 核心方法逐段讲解（按论文逻辑走）

> 读法建议：把这篇论文当作“评测模块的工程设计文档”，而不是模型训练论文。你迁移到比赛时，主要收益来自：更好的验证指标、更稳定的调参目标、更清晰的诊断视角。

4.1 模块 A：ngrFβ 框架（chrF 与 WORDF 的统一底座）

直觉（为什么有效）

MT 评价本质是“匹配 + 容错”。Fβ 让你显式权衡两类错误：

- 漏译（missing content）→ recall 低
- 乱译/冗余（hallucinated/extra content）→ precision 低
  在信息型翻译里，漏译往往更伤，所以常取 β>1。

形式化（关键公式/伪代码，用代码块）

```
# Generic n-gram F_beta with averaged precision/recall over n=1..N for n in 1..N: P_n = match_count(hyp, ref, n) / total_count(hyp, n) R_n = match_count(hyp, ref, n) / total_count(ref, n) P = average(P_1..P_N) R = average(R_1..R_N) F_beta = (1+beta^2)*P*R / (beta^2*P + R)
```

工程细节（实现坑、默认超参、复杂度）

- 匹配计数一般用 clipped count（按 reference 出现次数截断）避免刷重复；论文正文未展开实现细节，【不确定】请以作者脚本实现为准。
- 复杂度近似 O(L·N)（L 为序列长度），CPU 计算也很快，适合训练时高频评估。

4.2 模块 B：β=2 的选择（为什么不是 1 或 3）

直觉（为什么有效）

β 越大越偏 recall，会更重视“覆盖参考内容”，但也更容易对输出长度宽容，从而诱导系统在调参/解码时“变长”。

形式化（关键点）

作者基于 DA 相关性复核：β=2 与 β=3 都明显优于 β=1，但二者差距很小；工程上取 β=2 更稳。

工程细节

- 风险提示：作者提到 β=3 的 chrF3 在某些系统调参/组合场景可能生成过长句子（too long sentences），这对比赛解码超参搜索尤其重要。
- 工程策略：使用 chrF++ 做调参目标时，必须同时监控“平均输出长度”和长度分布（见第 5 节）。

4.3 模块 C：哪些 n-gram 真正“决定成败”

直觉（为什么有效）

- char n-gram：中等长度（3、4）信息量更大，比单字符更少噪声，比长片段更不苛刻。
- word n-gram：短依赖（1、2）最稳；长依赖（3、4）对自由改写太敏感，容易错杀高质量输出。

形式化（你在实现上要做的决定）

- chrF：字符 n 取 1..6（默认）
- WORDF：词 n 的有效上限其实更像 1..2，而不是 1..4

工程细节

- 作者尝试过给不同 n 加权、组合等复杂操作，整体上并没有比“把 word3/4 去掉”更稳定的收益；工程上这意味着：不要过度设计权重，先做最小改动。

4.4 模块 D：chrF+ / chrF++ 的定义与组合方式

直觉（为什么有效）

chrF 负责“形态/拼写容错”，word1/2 负责“词选择与局部搭配约束”。两者合并能把指标从“偏乐观/偏悲观”的两端拉回中间，更贴 DA。

形式化（关键公式/伪代码）

论文的文字表述是：把 word n-grams 加到 char n-grams 中并“一起平均（averaged together）”。【不确定】正文未给出更显式公式，建议按最常见实现口径：

```
# chrF++ (recommended interpretation) # char n = 1..6, word n = 1..2 P = average(P_char_1..6, P_word_1..2) R = average(R_char_1..6, R_word_1..2) chrFpp = F_beta(P, R; beta=2)
```

chrF+ 则对应只加入 word unigram（word n=1）。

工程细节（实现坑、默认超参、复杂度）

- 默认超参（常见口径）：char n=1..6，word n=1..2，β=2
- 实现对齐：为了 100% 与论文一致，直接调用作者脚本是最低风险路径（见第 5 节）
- 复杂度：与 chrF 同量级，只多了 word1/2 计数，几乎可忽略

5. 训练/推理细节（工程读者最关心的部分）

> 这篇论文不训练模型，但它改变你“怎么选 checkpoint、怎么调 beam、怎么看诊断”。在 Kaggle 里，这往往比“换个更大模型”更划算。

数据处理：<tokenizer/归一化/噪声处理等>

作者强调 tokenisation-independent（不依赖分词）与 raw text 输入。工程上仍建议做两类规范化以降低评测噪声：

1. Unicode 规范化（Unicode normalization）：全角/半角、组合字符、不同引号等（【不确定】论文未写，但现实工程常见）
2. 空白规则统一（whitespace normalization）：多空格、换行、行尾空格；字符 n-gram 对空白敏感（【不确定】以脚本为准）

训练策略：<batching, lr, schedule, 正则, dropout...>

- 推荐用法：把 chrF++ 作为 dev 主指标之一，用于 early stopping / checkpoint selection
- 若比赛最终指标是 BLEU 与 chrF++ 的合成（例如几何均值），建议：
  - 训练仍用交叉熵（cross-entropy）稳定收敛
  - 选择 checkpoint 与解码超参时用“合成指标”做最终判据
- 关键：不要把 chrF++ 当作 loss 直接优化（不可导、工程复杂、收益不稳）

推理策略：<beam, length penalty, constraints...>

- 由于 β>1 偏 recall，配合 beam search 容易出现“长度膨胀”倾向
- 工程建议：
  - 必做：beam + length penalty 网格搜索（或更智能的超参搜索）
  - 必监控：平均输出长度、长度分位数、重复率
  - 必对齐：用与官方提交一致的后处理（detokenize、标点规则等）

复杂度与资源：<显存/时间/吞吐，能估就估>

- chrF++ 的计算轻量，CPU 足够；训练显存瓶颈不在评测指标
- 在 8GB 显存场景下，瓶颈通常是推理（beam）与模型本体；指标更像“让你更频繁做小实验”的加速器

6. 实验设计：他们怎么证明“不是玄学”

数据集与划分：<重点：是否有泄漏风险、group split 等>

- 数据来源：WMT-15 与 WMT-16 shared task 的系统输出
- 目标语言：English 与 Russian（两类方向：多语到英、英到俄）
- 验证方式：句级 Pearson correlation 对齐 DA（direct assessment）
- 风险：可用 DA 文本量有限，语言覆盖有限，作者也明确承认结论需要更多语言验证

Baseline：<有哪些对照>

- chrF（字符 n-gram Fβ）
- WORDF（词 n-gram Fβ）
- 不同 β（1/2/3）
- 不同 word N（1..4），以及组合版本 chrF+ / chrF++

消融：<哪几个结论最关键>

1. β 选择：β=2/3 明显优于 β=1，且 β=2 与 β=3 差距很小 → 取 β=2 更稳
2. 单项 n-gram：char3/4 与 word1/2 最强；word3/4 明显拖后腿
3. 组合：把 word1/2 加到 chrF 里 → chrF++ 最稳；同时 WORDF 从 N=4 降到 N=2 会显著改善（证明长 word n-gram 是主要问题）

结果解读：<不仅报数，还解释趋势>

- WORDF 的“过度悲观”：高 DA 的点被压低（错杀改写）
- chrF 的“更集中”：更贴 DA，但可能略乐观（字符相似不代表表达自然）
- chrF++ 的收益：把两者拉回中间——用 word1/2 纠偏 chrF 的乐观面，用 char n-gram 纠偏 WORDF 的苛刻面

失败案例：<如果论文有错误分析就提炼；没有就推断可能的失败类型并标注为推断>

论文缺少系统的错误分析，这里给工程推断（仅供你做诊断方向）：

- 【推断】“字符很像但词不对/搭配不自然”：chrF 可能偏高，chrF++ 会更敏感
- 【推断】“正确词但乱序”：word bigram 会惩罚，chrF 相对宽容；chrF++ 介于两者之间
- 【推断】“输出系统性偏长”：β 偏 recall + beam 可能放大长度倾向，需要 length penalty 控制

7. 批判性复盘：这篇论文哪些地方可能不靠谱

强依赖的前提

- DA 数据量与语言覆盖有限（仅 English/Russian），外推到更多语种需谨慎

- Pearson r 假设线性相关足够表达“更像人”（可能存在非线性效应）【推断】

可能的负迁移点

- 目标语言的“词边界”若不稳定（如中文），word n-gram 会引入不稳定性（作者对中文只保留 chrF）
- 若把 chrF++ 当作唯一调参目标，可能诱导长度策略偏移（尤其 β 偏 recall）

复现实验最容易踩的坑

- “tokenisation independent”不等于“输出随便写”：Unicode/空白/标点规则会导致指标抖动（建议规范化 + 与官方一致的后处理）
- 多参考（multiple references）如何合并匹配计数、micro/macro 的精确定义：论文正文未细写，【不确定】建议以作者脚本为准

与相邻方法的边界（“什么时候别用它”）

- 当你必须强惩罚长距离重排序、且参考译文风格高度统一：BLEU 或学习型指标可能更合适（但复杂度更高）
- 当目标语言“词”概念不清或分词边界不稳定：别用 chrF++，退回 chrF

8. 迁移到 Deep Past（Akkadian→English）的落地方案

> 你的任务是 Akkadian transliteration → English。chrF++ 评的是英文输出与英文参考，因此它直接影响你：如何选 checkpoint、如何调解码、如何做错误诊断，而与源语言是否“楔形/转写”无直接耦合。

我们的 pipeline 插入点：<tokenizer / model / training / decoding / postprocess / eval>

- tokenizer：对英文输出的 detokenization/标点/空白做统一（降低 chrF++ 抖动）
- training：用 chrF++（与 BLEU 并行）做 dev 指标，控制 early stopping 与 checkpoint selection
- decoding：用合成指标调 beam / length penalty，并加长度监控防止“为 recall 变长”
- postprocess：与官方评测一致的规范化（大小写/标点/数字格式等，依比赛脚本而定）【不确定】
- eval：用 chrF vs chrF++ 的差异定位错误类型（字符相似但词搭配差等）

最小可复现实验（MVP）

对照组设置

1. Baseline：当前模型 + 默认解码（beam 与 length penalty 取你们已有经验值）
2. 对照 A：固定模型，仅调 length penalty（观察 chrF++ 的长度敏感性）
3. 对照 B：固定模型，beam 网格搜索（beam ∈ {1,2,4,8}）
4. 记录指标：BLEU、chrF、chrF++；并记录平均输出长度/长度分位数

需要改动的代码/配置点（按模块列）

- Eval 模块：接入 chrF / chrF+ / chrF++ 计算（推荐直接用作者脚本以避免实现偏差）
- Decode 模块：支持 beam、length penalty、max_len 的网格搜索与统一日志
- Logging：保存每个 checkpoint 的 (BLEU, chrF, chrF++) + 输出长度统计 + 若干样例输出

验收指标（BLEU/chrF++/几何均值 + 额外诊断指标）

- 主：比赛官方指标（若为 BLEU 与 chrF++ 的几何均值则以其为主）
- 辅：chrF++ 与 BLEU 的帕累托前沿（Pareto frontier）
- 诊断：平均输出长度、重复率、数字/专名错误率（可手写规则粗诊断）

预计算力（8GB 场景）与降级方案

- 指标计算：CPU 轻量，不是瓶颈
- 推理瓶颈：beam 与模型本体
- 降级：
  1. 先用 beam=1/2 快速筛 checkpoint
  2. 只对 top-k checkpoint 进行大 beam 精调
  3. 超参搜索只用固定抽样的 dev 子集（固定 seed），保证可复现

进一步迭代路线（最多 5 条，按优先级）

1. 指标驱动解码调参：系统化网格/贝叶斯优化在官方合成指标上找最优 beam/length
2. 分桶诊断：按句长/专名密度/数字密度分桶比较 chrF 与 chrF++ 的偏好差异
3. “坏案例库”：收集高 chrF++ 但语义/可读性差的样例，建立约束性后处理或诊断标签
4. 输出规范化策略：统一引号、破折号、数字格式（谨防改变语义）
5. 【推断】n-best 重排序：在候选译文上做基于 chrF++ 的近似 MBR（minimum Bayes risk）重排序

9. 读完该做什么（行动清单）

立刻能做的 3 件事（1 小时内）

1. 把 chrF / chrF+ / chrF++ 的计算接入你们仓库 eval（优先直接用作者脚本，避免实现偏差）
2. 对现有模型输出同时计算 BLEU 与 chrF++，并记录平均长度，检查“长度-分数”耦合
3. 做一次小对比：WORDF(N=4) vs WORDF(N=2) vs chrF vs chrF++（用同一批输出），确认 word3/4 的“错杀效应”

需要半天的 3 件事

1. beam/length penalty 网格搜索，画出（BLEU, chrF++）散点，找 Pareto 前沿
2. 抽样 50~100 条做“指标-现象”对齐：高 chrF++ 低 BLEU 的句子通常是哪些错误
3. 统一输出规范（Unicode/空白/标点）并复跑，评估指标方差（variance）

需要 1~3 天验证的 3 件事

1. 做一个轻量“人工 DA 替身”(proxy DA)：两人盲评 100~200 句，验证 chrF++ 是否更贴你们的偏好排序
2. 分桶分析（句长/专名/数字）：定位 chrF++ 的强项与盲点，决定是否加诊断指标
3. 将 checkpoint 选择策略从“单指标最优”升级为“官方合成指标最优 + 长度约束”，降低指标过拟合风险